

Data-Driven Compute Overlays for Interactive Geographic Simulation and Visualization

Patrick Komon*
TU Wien

Gerald Kimmersdorfer†
TU Wien

Adam Celarek‡
TU Wien

Manuela Waldner§
TU Wien

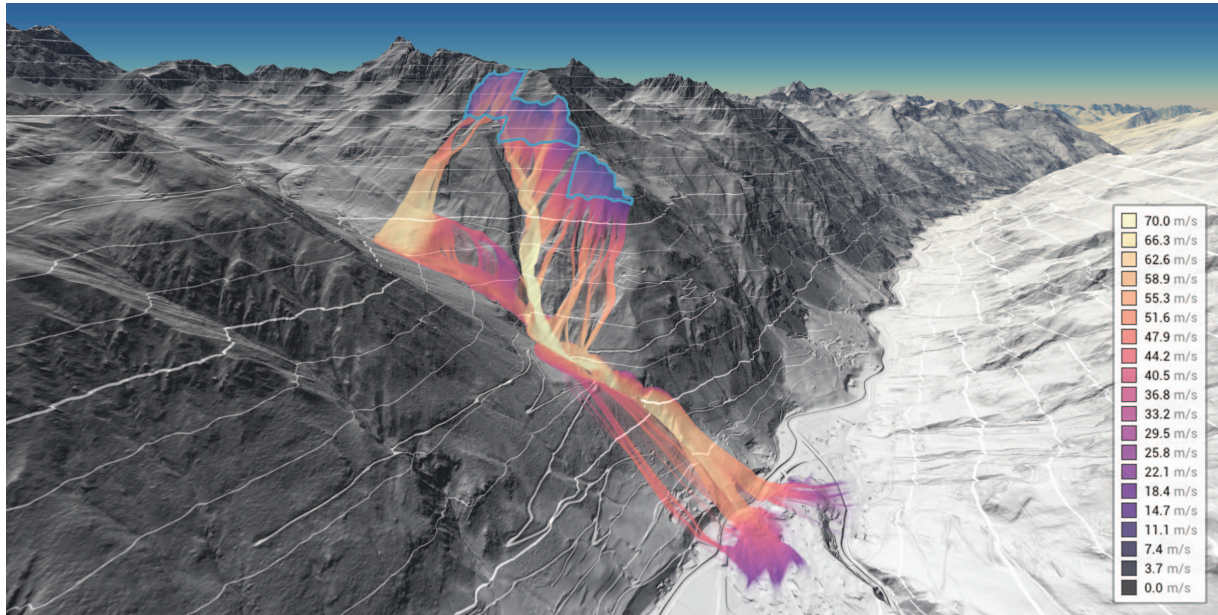


Figure 1: Simulation of the Madlein avalanche (Tyrol, 1984) from its recorded release areas (highlighted in blue), computed and visualized in a few milliseconds. The color encodes the velocity.

ABSTRACT

We present interactive data-driven compute overlays for native and web-based 3D geographic map applications based on WebGPU. Our data-driven overlays are generated in a multi-step compute workflow from multiple data sources on the GPU. We demonstrate their potential by showing results from snow cover and avalanche simulations, where simulation parameters can be adjusted interactively and results are visualized instantly. Benchmarks show that our approach can compute large-scale avalanche simulations in milliseconds to seconds, depending on the size of the terrain and the simulation parameters, which is multiple orders of magnitude faster than a state-of-the-art Python implementation.

Index Terms: 3D geographic visualization, geographic simulation, WebGPU.

1 INTRODUCTION

Data-driven overlays for geographic maps showing, for instance, steepness of the terrain, precipitation, or points of interest, are typically computed offline. This limits the applicability to static data. To overlay 2D or 3D maps with visualizations of dynamic data

or results of interactively steerable simulations, the overlays either need to be limited to a very small area or require remote access to a powerful server.

In this work, we explore the use of WebGPU to compute data-driven overlays for online maps dynamically with the support of compute shaders. We present a workflow for running simulations and instantly generating overlays for 3D maps and demonstrate how our prototype *weBIGeo* can speed up conventional workflows by orders of magnitude. In summary, our contributions are as follows:

1. An API to specify compute workflows for instant generation of data-driven multi-resolution overlays for 3D maps.
2. The first open-source 3D geographic simulation and visualization tool *weBIGeo* for interactive native and web-based applications with compute shader support,
3. Two use cases – snow cover and avalanche simulation (Fig. 1) – demonstrating the potential of data-driven overlays.

Our source code is available on [GitHub](https://github.com). An online demo showcasing the avalanche simulation use case can be accessed here: <https://webigeo.alpinemaps.org/> (WebGPU-capable browser required, for example Google Chrome on Windows).

2 BACKGROUND: (WEB-BASED) GEOVIS

Common geographic visualization techniques are glyphs placed on top of maps and color overlays [12]. The focus of our work lies on overlays. Here, we review three principle paradigms to data-driven map overlays particularly for the web:

*e-mail: pkomon@cg.tuwien.ac.at

†e-mail: gkimmersdorfer@cg.tuwien.ac.at

‡e-mail: celarek@cg.tuwien.ac.at

§e-mail: waldner@cg.tuwien.ac.at

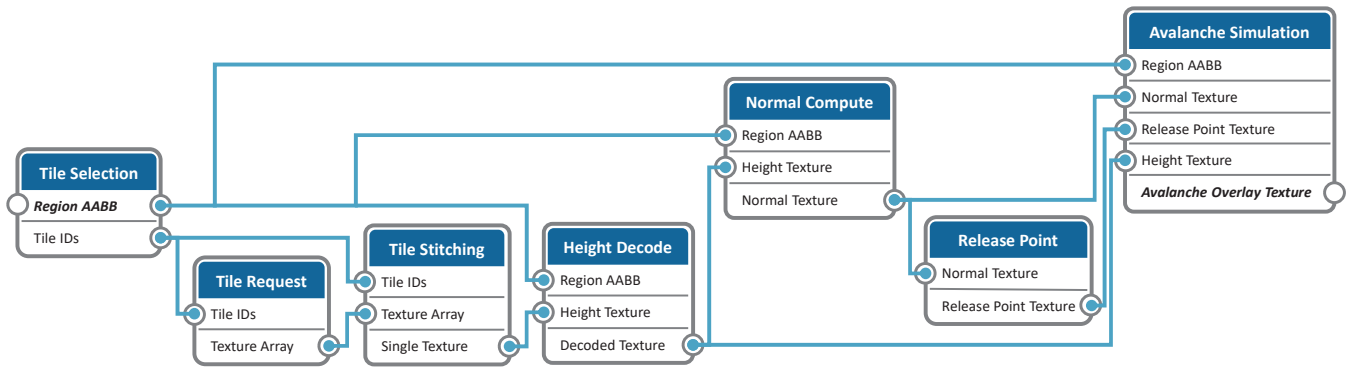


Figure 2: A node graph generated from the workflow specification for the avalanche use case (Sec. 5.2) This workflow generates a multi-resolution *Avalanche Overlay Texture* (right) from terrain model tiles in a given *Region AAB* (left).

Map tiles are the foundation for all popular online map applications, where map clients request pre-rasterized map images depending on the currently shown zoom level and visible field of view [13] from a dedicated server – also known as “slippy maps” [8]. Data-driven overlays can be similarly pre-rendered into **offline tiles** and overlaid onto a base map [6, 15]. A well-known open-source library to build layered maps is OpenLayers [16]. Offline tiles are useful when the data to be overlaid itself is big and static. The disadvantage is that the server has to re-compute the tiles whenever the underlying data changes. Users have no or only very limited possibilities to interact with the data.

An alternative approach is to render the final images to be viewed by the client directly on the server. The client only forwards users’ interaction with the map (e.g., panning and zooming) and with the data (e.g., filtering) to the server and finally shows the image transmitted by the server. The application is therefore largely independent from the client’s hardware resources. In the geographic context, examples for such a **server-side analysis and rendering** approach are HORA 3D for interactive flooding simulations [14] and applications built on top of ParaView [2]. The disadvantage is that the server needs to be powerful enough to serve all requests at the same time. In addition, interactive applications require a lot of bandwidth for smooth interaction, such as flying over the 3D terrain in Fig. 1.

The most flexible approach is **client-side analysis and rendering**: the data to be analyzed remains on the client, and the client handles all interactivity, simulation, and rendering itself. The major disadvantage, clearly, is that the client needs to be powerful enough – in terms of CPU, GPU, and memory – to handle the computation of the overlays and all user interactivity. Client-side overlays can be computed pixel-wise for every frame in the fragment shader based on the raster tiles used for rendering the 3D map. Examples are risk maps computed from local terrain properties [7] or difference maps of flood simulation scenarios [9]. There are some major shortcomings to this approach: First, overlays need to be recomputed whenever user navigation requires a new map tile to be loaded, even if the underlying data does not change. Second, the resolution of the overlay is bound to the resolution of the underlying map tile. Third, pixel-wise analysis is not sufficient for more complex scenarios like avalanche simulation (see Sec. 5.2). It has been shown that some more complex geospatial analyses can be significantly sped up by GPGPU, using, for instance OpenCL [4] or CUDA [22]. Vollmer et al. [20] compute levels-of-detail of pre-computed static overlay textures on the GPU so that the user can interactively change aggregation parameters. In contrast, we propose a workflow and present an implementation to interactively compute overlay textures directly from the raw data.

Up until recently, graphics APIs for the web were limited to ver-

tex and fragment shaders. With the new web-based graphics and compute GPU API *WebGPU*, it is now possible to employ compute shaders in the browser. WebGPU is increasingly generating interest for interactive web-based visualization of large data. In particular, it has been used for interactive isosurface extraction [18], graph drawing [5], and aggregate 2D GeoVis [11]. WebGPU is generally seen as a performant successor of WebGL in the context of Web-GIS applications [19] and is appreciated for its code portability between web, desktop, and mobile map applications [1]. We extend these works by contributing a workflow, a software architecture, and performance benchmarks for data-driven compute overlays for 3D maps. To the best of our knowledge, *weBIGeo* is thereby the first online map with interactively computed data-driven overlays.

3 DATA-DRIVEN COMPUTE OVERLAYS

weBIGeo exposes a simple and flexible API to construct compute workflows of varying complexity. Compute workflows are specified as a sequence of compute nodes, similarly to dataflow models for visualization provenance [17] or for steering simulation ensembles [21]. Each compute node has at least one input and one output, as illustrated in the node graph in Fig. 2. Raw input data can come in various forms – from single values, such as the current snow line in the region of interest, over spatial point data stored in a spatial data structure, such as the location of weather stations stored in a kd-tree, to raster data stored in a tile format, such as a digital elevation model (DEM) capturing the ground elevation within a given region. The output of a compute step can be resources like textures, buffers, or vectors. The compute workflow can be configured such that the output of one compute step can serve as input to a subsequent one. This allows arbitrarily reusing and composing compute steps across multiple compute workflow definitions. Compute nodes themselves can represent any CPU or GPU computation. If the input data can be decomposed into small, independent units, a node may execute a compute shader to process each unit in parallel.

weBIGeo employs a multi-pass rendering pipeline with deferred shading. In the geometry pass, it draws the z-buffer of the base map with albedo, position, and normal. In the compute pass, the multi-resolution overlay texture is generated, by executing the specified compute workflow. In the final compose pass, we first set an atmosphere background, then apply Phong shading to the base map, and finally alpha-blend the data-driven overlay with the base map at its specified location.

A major advantage of compute overlays in contrast to an approach that computes overlays in the geometry pass using a fragment shader is that the compute pass only needs to be re-executed when the input data changes or the user has modified simulation parameters. Another advantage is that data-driven overlays can be computed on the finest reasonable zoom level of the input data, in-

dependently of the base map resolution, which depends on the camera distance and is therefore not uniform across the screen for 3D maps. Computing overlays in map resolution may lead to artifacts and information loss, as we will illustrate in Sec. 5.1. From the high-resolution overlay texture, we then compute a mipmap, which represents the data-driven, multi-resolution overlay.

A disadvantage of compute overlays is that the entire high-resolution overlay texture needs to be computed in a single compute pass. This implies that the extent of the region for which to compute the overlay is bound by the available client resources, most importantly GPU memory and maximum supported texture size. In our implementation, we therefore limit the texture size to WebGPU's current default (minimum) limit of 8192×8192 pixels to ensure compatibility with all devices supporting WebGPU.

4 IMPLEMENTATION

Conceptually, the data-driven compute overlay API of *weBIGeo* could be implemented on top of any graphics API or rendering engine that supports compute shaders. With WebGPU, we now for the first time have the opportunity to provide client-side compute overlays for web-based map applications.

For rendering the 3D base map, *weBIGeo* uses a traditional tile-based map rendering approach. As input, it requests tiles from a DEM to compute the 3D terrain mesh, as well as corresponding orthophotos as textures. At the moment, *weBIGeo* is limited to Austria and uses open data provided by basemap.at.

weBIGeo is written in C++ on top of our 3D map rendering framework AlpineMaps.org that takes care of camera handling, tile scheduling, and caching. We additionally use Qt for handling multi-threading, networking, file I/O etc., and ImGui for the user interface. *weBIGeo* can be built as a native Windows or Linux application (with Google's Dawn backend), or as a web application (using the Emscripten compiler to convert the C++ code into WebAssembly format, which is supported by most of today's browsers).

5 USE CASES

We show two proof-of-concept compute workflows to demonstrate the potential and flexibility of data-driven compute overlays. For both use cases, we briefly discuss the compute workflow nodes, the simulation parameters, the data decomposition strategy, and the results. For the first example, we draw a visual comparison between an overlay computed in the fragment shader in base map resolution and our data-driven compute overlays. The second example shows the results of avalanche simulations, which cannot be computed in a conventional fragment shader approach. We report performance benchmarks for this use case and compare the results to a state-of-the-art avalanche simulation without GPU support.

5.1 Snow Cover

The first use case is a simple snow cover simulation, which predicts snow cover solely from terrain features and thereby can easily achieve a fast visual effect for 3D maps. Through the GUI, the user can interactively control a few parameters, such as the altitude of the snow line, the maximum steepness of the terrain, as well as blending factors for the transition areas. The snow cover overlay renders snow-covered areas in white, while the alpha corresponds to the blending factor.

The first five steps of the compute workflow are identical to the workflow for simulating avalanches, discussed in Sec. 5.2 and shown in Fig. 2: Within a desired region bounding box, we retrieve corresponding high-resolution (approximately 1×1 meters) tiles of the DEM, stitch the tiles into one DEM texture, and compute the surface normals. The final compute node then decides – depending on the given input parameters – whether to set the albedo of the base map to white or whether to pick the color of the corresponding orthophoto texture. The normals and final snow computation

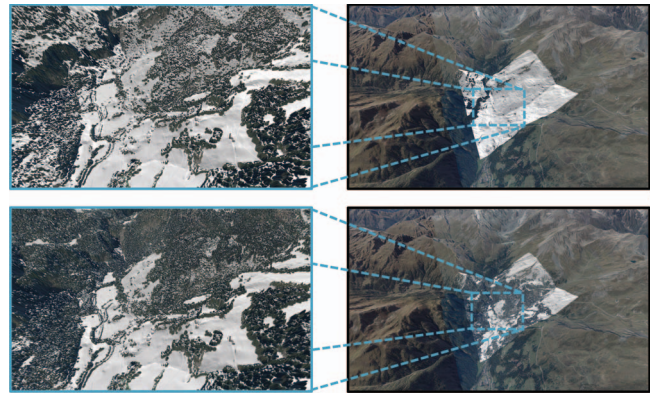


Figure 3: Simple snow cover simulation using a fragment shader on top of the base map (top) or using a data-driven compute overlay (bottom). When zooming out (right), the fragment shader approach shows too much snow on top of forests and other locally steep terrain (top right). Zoom in for more details.

can both be easily parallelized for each texel of the DEM texture. Given WebGPU's texture size limitations, we can currently cover a region of 2048×2048 meters ($\approx 4.2\text{km}^2$) requiring 256 MiB of GPU memory.

Since the terrain is only evaluated locally for this snow effect, it could theoretically also be computed in a fragment shader in each base map rendering step based on the DEM geometry. Fig. 3 (top) illustrates the limitations of this approach: by coupling the snow cover simulation to the currently rendered base map resolution, the visual quality derogates significantly as the user zooms out. This happens because the normal vectors, based on which the steepness is evaluated, are flattened in map resolution as the distance to the camera increases, and more coarse DEM tiles are loaded. Thus, regions with large variety in steepness from the DEM, such as forests, will be interpreted as flat and rendered in white from distance. In contrast, the data-driven snow cover overlay is computed on the highest available resolution of the DEM. In the compose pass, the renderer then interpolates the overlay texture rather than the normals. This way, we can better preserve the impression of sparse snow cover in regions with high variation in steepness (Fig. 3 bottom).

5.2 Avalanches

Gravitational mass flow (GMF) models compute the flow of material under gravity in a given terrain and are commonly used for snow avalanche, landslide, or rock fall simulations. Physically-based GMF models are computationally too expensive for larger-scale simulations and are also very sensitive to input parameters. Larger-scale simulations therefore usually rely on empirical models that use fewer input parameters and can approximate GMF to a certain degree. A state-of-the-art model is FlowPy [3], which is implemented in Python and which we will use for comparison.

As shown in Fig. 2, the compute workflow first fetches elevation tiles of the DEM in the region of interest and computes the normals. By default, we operate on the recommended DEM resolution for avalanche simulations of five to ten meters [10]. The subsequent release point node computes, for each texel of the normal texture in parallel, whether the point is a potential release point of an avalanche. Our release area selection is a very simple approximation, evaluating the local steepness of the terrain as only input parameter, thereby functioning very similarly as the previously described snow cover simulation. For this compute step, the user can control in which texture resolution to evaluate potential release points and which steepness range to consider. More sophisticated

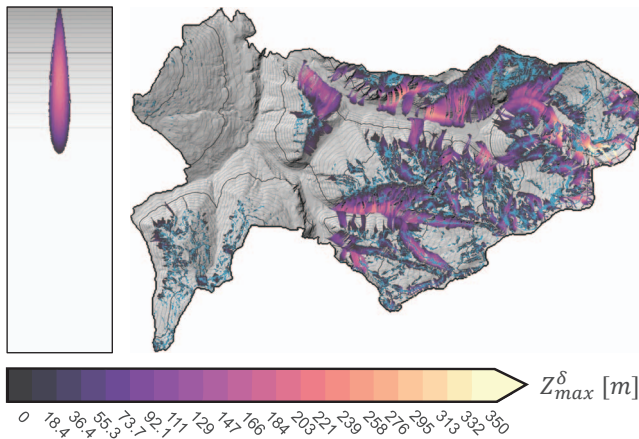


Figure 4: *weBIGeo* avalanche simulation results on the *parabola* (left) and *vals* (right) datasets. Release points are highlighted in blue. Contour lines represent the terrain, spaced at 50m intervals. Color encodes the maximum vertical displacement per step Z_{max}^{δ} , which can be interpreted as an indicator for velocity. Zoom in for more details.

models take further parameters – like the upslope area to model the water input or geological maps describing sediment availability – into consideration [10]. Alternatively, we can provide known release points encoded in a pre-computed texture, such as for historic avalanche events, as shown in Fig. 1. The compute node outputs a binary map indicating whether avalanche particles should be triggered from this point or not.

The final compute node models the actual avalanche particle trajectories and the runout distance. It takes the release point texture and the DEM as input, together with a variety of user-defined simulation parameters, such as the ratio of persistence, which simulates some sort of momentum, and runout angle, which is a simple stopping criterion based on the angle between the current position and the release point of the particle. The user furthermore controls the number of particles released per coded texel from the release point texture. To approximate the chaotic processes within an avalanche, we add a user-defined amount of random variation from the optimal path of a particle. With this Monte-Carlo approach, we can then simulate spreading mass along the flow. The avalanche trajectory node computes the trajectories in parallel for all released avalanche particles.

To compare the computational runtime between FlowPy’s [3] Python implementation and our WebGPU compute overlays, we obtained runtime measures on two client devices: A high-end consumer *desktop* (32 GB RAM, AMD Ryzen 9 3900X, NVIDIA RTX 3070 with 8 GB memory) and a commodity *notebook* (16 GB RAM, Intel(R) Core(R) i7-1165G, integrated GPU with 2GB memory). For FlowPy, we used 16 CPU cores for *desktop* and four cores for *notebook* and fixed the parameter *exp*, which controls the amount of deviation from a single-directional flow, to 8. We performed the evaluation on two data sets: *Parabola* is a synthetic dataset with a parabolic slope with a resolution of 501×151 input cells, three of which are marked as release cells. *Vals* is a large-scale real-world data set from Tyrol, covering an area of more than 100km^2 . For our tests, simulation parameters from both data sets were read back to the CPU. We show one of these parameters in Fig. 4. The data sets are described in more detail in the FlowPy paper [3].

We empirically chose parameters *persistence* as 90% and *randomness* as 16% to approximate the visual results of FlowPy using an *exp* parameter of 8. Increasing *randomness* introduces spread

Table 1: Model runtime comparison of FlowPy [3] vs *weBIGeo*, including speedup factor of *weBIGeo* over FlowPy.

Hardware	Dataset	FlowPy	<i>weBIGeo</i>	Speedup
<i>desktop</i>	<i>parabola</i>	10,1 s	13,5 ms	748×
	<i>vals</i>	78,0 min	1,50 s	3120×
<i>notebook</i>	<i>parabola</i>	8,19 s	41,6 ms	197×
	<i>vals</i>	266,7 min	6,74 s	2374×

and requires computing more particles to achieve sufficient coverage. We found that using 2048 particles per release cell provides good visual results for the chosen randomness. All *weBIGeo* performance reports were averaged from ten runs.

A comparison between the average model runtimes shows significant speedup of *weBIGeo* over FlowPy (Tab. 1). On both hardware setups, *weBIGeo* takes milliseconds for *parabola* and seconds for *vals*. In contrast, FlowPy performs in the order of seconds for *parabola* and hours for *vals*. These timings exclude file read and write times. As *weBIGeo* allows to visualize the simulation output directly, file export is only performed on-demand.

6 DISCUSSION & CONCLUSIONS

Our work provides a first proof-of-concept and evaluation to assess the potential and limitations of WebGPU in the context of geographic simulation and visualization. Our first use case has shown that data-driven compute overlays can notably increase the visual quality compared to a fragment shader approach due to the independence of the overlay resolution from the base map. Our second use case shows that data-driven compute overlays have the potential to speed up gravitational mass flow simulations by orders of magnitude compared to the state-of-the-art implemented on the CPU. We have furthermore shown that client-side analysis and rendering is a feasible option even for commodity notebook hardware using WebGPU. The main limitation is that – at least for overlays computed on top of high-resolution digital elevation models as in our use cases – the spatial extent of the overlay is limited by the GPU memory.

While our use cases in this paper focus on interactive simulations, the principle of data-driven compute overlays can also be applied to interactive visualizations of large geographic data in the future. For instance, a compute workflow could generate heatmap overlays showing millions of user-selected points of interest or visualize the solar potential in an area of interest under different interactive scenarios.

The choice whether to rely on client-side analysis and rendering or alternative approaches, such as pre-computed offline tiles or server-side analysis and rendering, finally depends on multiple aspects, such as the amount of data to be analyzed, the resources of client and server, the number of anticipated parallel users, the provided interactivity, and the bandwidth. We believe that client-side analysis and rendering is particularly promising for expert user scenarios, where we can expect users to be equipped with relatively powerful devices and a strong desire to interact intensively with simulation parameters. In the future, we therefore see WebGPU-based data-driven compute overlays as interesting option for geographic data analysis and visualization, opening the possibility for real-time inspection of simulation parameters and large geographic data for many different scientific and public domains.

ACKNOWLEDGMENTS

We thank Jan-Thomas Fischer, Felix Oesterle, Paula Spanning, and Markus Rampp for their feedback and theoretical input for the avalanche simulations. This project was funded by NetIdee (project 6745).

REFERENCES

- [1] M. Ammann, A. Drabble, J. Ingensand, and B. Chapuis. Maplibre: Toward portable map renderers. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLVIII-4/W1-2022:35–42, 2022. doi: 10.5194/isprs-archives-XLVIII-4-W1-2022-35-2022 2
- [2] A. Cedilnik, B. Geveci, K. Moreland, J. P. Ahrens, and J. M. Favre. Remote Large Data Visualization in the ParaView Framework. In *Eurographics Symposium on Parallel Graphics and Visualization*, pp. 163–170. The Eurographics Association, Goslar, 2006. doi: 10.2312/EGPGV/EGPGV06/163-170 2
- [3] C. J. L. D’Amboise, M. Neuhauser, M. Teich, A. Huber, A. Kofler, F. Perzl, R. Fromm, K. Kleemayr, and J.-T. Fischer. Flow-Py v1.0: a customizable, open-source simulation tool to estimate runoff and intensity of gravitational mass flows. *Geoscientific Model Development*, 15(6):2423–2439, 2022. doi: 10.5194/gmd-15-2423-2022 3, 4
- [4] S. Dübel, M. Röhlig, C. Tominski, and H. Schumann. Visualizing 3D Terrain, Geo-Spatial Data, and Uncertainty. *Informatics*, 4(1):6, 2017. doi: 10.3390/informatics4010006 2
- [5] L. Dyken, P. Poudel, W. Usher, S. Petruzza, J. Y. Chen, and S. Kumar. GraphWaGu: GPU Powered Large Scale Graph Layout Computation and Rendering for the Web. In *Eurographics Symposium on Parallel Graphics and Visualization*. The Eurographics Association, 2022. doi: 10.2312/pgv.20221067 2
- [6] M. J. Egenhofer and J. R. Richards. Exploratory access to geographic data based on the map-overlay metaphor. *Journal of Visual Languages & Computing*, 4(2):105–125, 1993. doi: 10.1006/jvlc.1993.1009 2
- [7] J. Eschner, A. Celarek, and M. Waldner. Echtzeitvisualisierung von Lawinenrisiko basierend auf hochauflösenden Geodaten. In *Lawinen-symposium 2023*, pp. 38–43. GeoSphere Austria, Graz, Nov. 2023. 2
- [8] M. Haklay and P. Weber. OpenStreetMap: User-Generated Street Maps. *IEEE Pervasive Computing*, 7(4):12–18, 2008. doi: 10.1109/MPRV.2008.80 2
- [9] M. Heitzler, J. C. Lam, J. Hackl, B. T. Adey, and L. Hurni. GPU-Accelerated Rendering Methods to Visually Analyze Large-Scale Disaster Simulation Data. *Journal of Geovisualization and Spatial Analysis*, 1(3):1–18, 2017. doi: 10.1007/s41651-017-0004-4 2
- [10] P. Horton, M. Jaboyedoff, B. Rudaz, and M. Zimmermann. Flow-R, a model for susceptibility mapping of debris flows and other gravitational hazards at a regional scale. *Natural hazards and earth system sciences*, 13(4):869–885, 2013. doi: 10.5194/nhess-13-869-2013 3, 4
- [11] G. Kimmersdorfer, D. Wolf, and M. Waldner. WebGPU for Scalable Client-Side Aggregate Visualization. In *EuroVis 2023 - Posters*, pp. 105–107. The Eurographics Association, June 2023. doi: 10.2312/evp.20231079 2
- [12] A. Luo, D. Kao, and A. Pang. Visualizing spatial distribution data sets. In *Proceedings of the Symposium on Data Visualisation 2003*, vol. 3, pp. 29–38. The Eurographics Association, Goslar, 2003. 1
- [13] S. Quinn and M. Gahegan. A predictive model for frequently viewed tiles in a web map. *Transactions in GIS*, 14(2):193–216, 2010. doi: 10.1111/j.1467-9671.2010.01191.x 2
- [14] S. Rauer-Zechmeister, D. Cornel, B. Sadransky, Z. Horváth, A. Konev, A. Buttinger-Kreuzhuber, R. Heidrich, G. Blöschl, E. Gröller, and J. Waser. HORA 3D: Personalized Flood Risk Visualization as an Interactive Web Service. *Computer Graphics Forum*, 43(3):e15110, 2024. doi: 10.1111/cgf.15110 2
- [15] M. Reddy, Y. Leclerc, L. Iverson, and N. Bletter. TerraVision II: Visualizing massive terrain databases in VRML. *IEEE Computer Graphics and Applications*, 19(2):30–38, 1999. doi: 10.1109/38.749120 2
- [16] A. Santiago. *The book of OpenLayers 3: Theory & Practice*. Leanpub, Victoria, BC, 2015. 2
- [17] C. T. Silva, J. Freire, and S. P. Callahan. Provenance for Visualizations: Reproducibility and Beyond. *Computing in Science & Engineering*, 9(5):82–89, 2007. doi: 10.1109/MCSE.2007.106 2
- [18] W. Usher and V. Pascucci. Interactive Visualization of Terascale Data in the Browser: Fact or Fiction? In *2020 IEEE 10th Symposium on Large Data Analysis and Visualization (LDAV)*, pp. 27–36. IEEE, 2020. doi: 10.1109/LDAV51489.2020.00010 2
- [19] Z. Usta. Webgpu: A new graphic api for 3d webgis applications. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLVIII-4/W9-2024:377–382, 2024. doi: 10.5194/isprs-archives-XLVIII-4-W9-2024-377-2024 2
- [20] J. O. Vollmer, M. Trapp, H. Schumann, and J. Döllner. Hierarchical Spatial Aggregation for Level-of-Detail Visualization of 3D Thematic Data. *ACM Transactions on Spatial Algorithms and Systems (TSAS)*, 4(3):1–23, 2018. doi: 10.1145/3234506 2
- [21] J. Waser, H. Ribicic, R. Fuchs, C. Hirsch, B. Schindler, G. Blöschl, and E. Gröller. Nodes on Ropes: A Comprehensive Data and Control Flow for Steering Ensemble Simulations. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):1872–1881, 2011. doi: 10.1109/TVCG.2011.225 2
- [22] Y.-j. Xia, L. Kuang, and X.-m. Li. Accelerating geospatial analysis on GPUs using CUDA. *Journal of Zhejiang University SCIENCE C*, 12(12):990–999, 2011. doi: 10.1631/jzus.C1100051 2